AI ASSISTED CODING

ASSIGNMENT -10.1

Name: Hari Priya

H.T.No: 2303A51104

Task Description #1 – Syntax and Logic Errors

Task: Use AI to identify and fix syntax and logic errors in a faulty Python script.

Sample Input Code:

Calculate average score of a student

def calc_average(marks):

total = 0

for m in marks:

total += m

average = total / len(marks)

return avrage # Typo here

marks = [85, 90, 78, 92]

print("Average Score is ", calc_average(marks)

Expected Output:

• Corrected and runnable Python code with explanations of the fixes.

```python
# refactored the below code to fix the typo and syntax error
def calc_average(marks):
    total = 0
    for m in marks:
        total += m
    average = total / len(marks)
    return average # Fixed typo: 'avrage' to 'average'
marks = [85, 90, 78, 92]
print("Average Score is ", calc_average(marks))
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS powershell

```
PS C:\Users\HARI PRIYA\Desktop\AIAC> & "C:\Users\HARI PRIYA\AppData\Local\Microsoft\WindowsApps\python3.11.exe" "c:/Users/HARI PRIYA/Desktop/AIAC/lab10.py"
Average Score is  86.25
```

Typo corrected – Changed return avrage to return average.

Missing bracket fixed – Added the closing) in the print() statement.

Task Description #2 – PEP 8 Compliance

Task: Use AI to refactor Python code to follow PEP 8 style guidelines.

Sample Input Code:

def area_of_rect(L,B) : return L*B

print(area_of_rect(10,20))

Expected Output:

• Well-formatted PEP 8-compliant Python code.

```python
#refactored the above code to enhance readability and maintainability well commented and add type hints
def area_of_rectangle(length: float, breadth: float) -> float:
    """
    This function calculates the area of a rectangle given its length and breadth.

    Parameters:
    length (float): The length of the rectangle
    breadth (float): The breadth of the rectangle

    Returns:
    float: The area of the rectangle
    """
    return length * breadth
# Example usage
length = 10.0
breadth = 20.0
area = area_of_rectangle(length, breadth)
print(f"The area of the rectangle with length {length} and breadth {breadth} is {area}.")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS powershell

```
PS C:\Users\HARI PRIYA\Desktop\AIAC> & "C:\Users\HARI PRIYA\AppData\Local\Microsoft\WindowsApps\python3.11.exe" "c:/Users/HARI PRIYA/Desktop/AIAC
0.py"
The area of the rectangle with length 10.0 and breadth 20.0 is 200.0.
```

Task Description #3 – Readability Enhancement

Task: Use AI to make code more readable without changing its logic.

Sample Input Code:

def c(x,y):

return x*y/100

a=200

b=15

print(c(a,b))

Expected Output:

• Python code with descriptive variable names, inline comments, and clear formatting.

```python
def calculate_percentage(part: float, whole: float) -> float:
    """
    This function calculates the percentage of a part relative to a whole.

    Parameters:
    part (float): The portion or part of the whole
    whole (float): The total or whole amount

    Returns:
    float: The percentage of the part relative to the whole
    """
    return (part * 100) / whole
# Example usage
numerator = 200
denominator = 15
percentage = calculate_percentage(numerator, denominator)
print(f"The percentage of {numerator} relative to {denominator} is: {percentage}%")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS powershell

```
PS C:\Users\HARI PRIYA\Desktop\AIAC> & "C:\Users\HARI PRIYA\AppData\Local\Microsoft\WindowsApps\python3.11.exe" "c:/Users/HARI PRIYA/Desktop/AIAC/
0.py"
The percentage of 200 relative to 15 is: 1333.3333333333333%
```

Task Description #4 – Refactoring for Maintainability
Task: Use AI to break repetitive or long code into reusable functions.
Sample Input Code:
students = ["Alice", "Bob", "Charlie"]
print("Welcome", students[0])
print("Welcome", students[1])
print("Welcome", students[2])
Expected Output:
• Modular code with reusable functions.

```python
# Break repetitive or long code into reusable functions. Modular code with reusable function
def welcome_students(student_list):
    """
    This function takes a list of student names and prints a welcome message for each student.

    Parameters:
    student_list (list): A list of student names

    Returns:
    None
    """
    for student in student_list:
        print(f"Welcome, {student}!")
# Example usage
students = ["Alice", "Bob", "Charlie", "David"]
welcome_students(students)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS powershell

```
PS C:\Users\HARI PRIYA\Desktop\AIAC> & "C:\Users\HARI PRIYA\AppData\Local\Microsoft\WindowsApps\python3.11.exe" "c:/Users/HARI PRIYA/Desktop/AIAC/
0.py"
Welcome, Alice!
Welcome, Bob!
Welcome, Charlie!
Welcome, David!
```

Task Description #5 – Performance Optimization
Task: Use AI to make the code run faster.
Sample Input Code:
# Find squares of numbers
nums = [i for i in range(1,1000000)]
squares = []
for n in nums:
squares.append(n**2)
print(len(squares))
Expected Output:
• Optimized code using list comprehensions or vectorized operations.

```python
import time
time1 = time.time()
nums = [i for i in range(1000000)]
squares = []
for n in nums:
    squares.append(n ** 2)
print(len(squares))
time2 = time.time()
# refactor the above code with less time complexity
time3 = time.time()
squares = [n ** 2 for n in range(1000000)]
print(len(squares))
time4 = time.time()
print(f"Time taken for original code: {time2 - time1} seconds")
print(f"Time taken for refactored code: {time4 - time3} seconds")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS powershell

```
PS C:\Users\HARI PRIYA\Desktop\AIAC> & "C:\Users\HARI PRIYA\AppData\Local\Microsoft\WindowsApps\python3.11.exe" "c:/Users/HARI PRIYA/Desktop/AIA
0.py"
1000000
1000000
Time taken for original code: 0.16692543029785156 seconds
Time taken for refactored code: 0.10420489311218262 seconds
```

Task Description #6 – Complexity Reduction
Task: Use AI to simplify overly complex logic.
Sample Input Code:
def grade(score):
if score >= 90:
return "A"
else:
if score >= 80:
return "B"
else:
if score >= 70:
return "C"
else:
if score >= 60:
return "D"
else:
return "F"
Expected Output:
• Cleaner logic using elif or dictionary mapping.

```python
# simplify below code with using elif statement
def grade(score):
    if score >= 90:
        return 'A'
    elif score >= 80:
        return 'B'
    elif score >= 70:
        return 'C'
    elif score >= 60:
        return 'D'
    else:
        return 'F'
print(grade(95))  # Output: A
print(grade(85))  # Output: B
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS powershell

```
PS C:\Users\HARI PRIYA\Desktop\AIAC> & "C:\Users\HARI PRIYA\AppData\Local\Microsoft\WindowsApps\python3.11.exe" "c:/Users/HARI PRIYA/Desktop/AIAC
0.py"
A
B
```

```python
# Using dictionary lookup
def grade(score):
    grades = {
        range(90, 101): 'A',
        range(80, 90): 'B',
        range(70, 80): 'C',
        range(60, 70): 'D',
    }
    for (low, high), letter in grades.items():
        if score in range(low, high):
            return letter
```